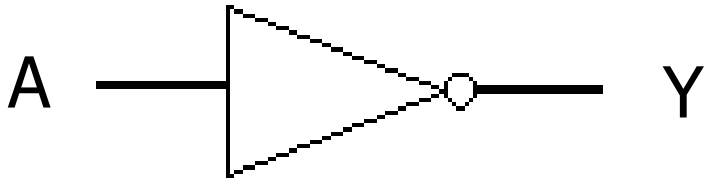


Digital Engineering

Boolean Algebra and Logic Gates

Describing Circuit Functionality: Inverter

- **Basic logic functions have symbols**
- **The same functionality can be represented with a truth table**
  - **Truth table completely specifies outputs for all input combinations**
- **This is an inverter**
  - **An input of 0 is inverted to a 1**
  - **An input of 1 is inverted to a 0**



Symbol

Truth Table

A	Y
0	1
1	0

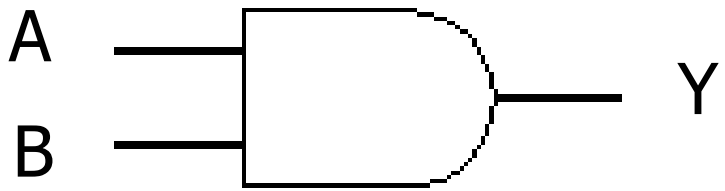
Input **Output**

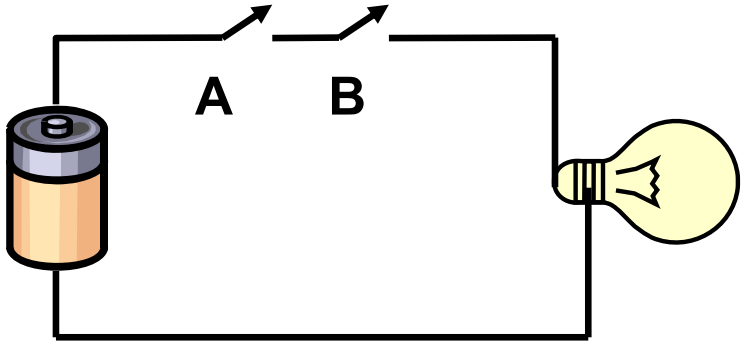
The AND Gate

- This is an **AND** gate
- If the two input signals are *asserted* (i.e. high) the output will also be asserted. Otherwise, the output will be *deasserted* (i.e. low)

Truth Table

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

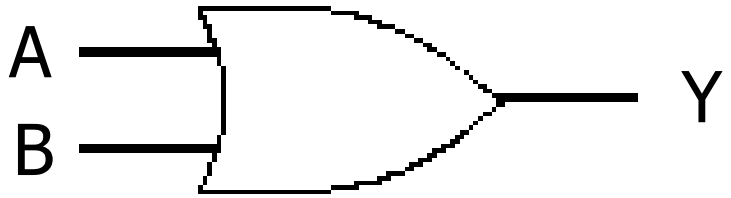


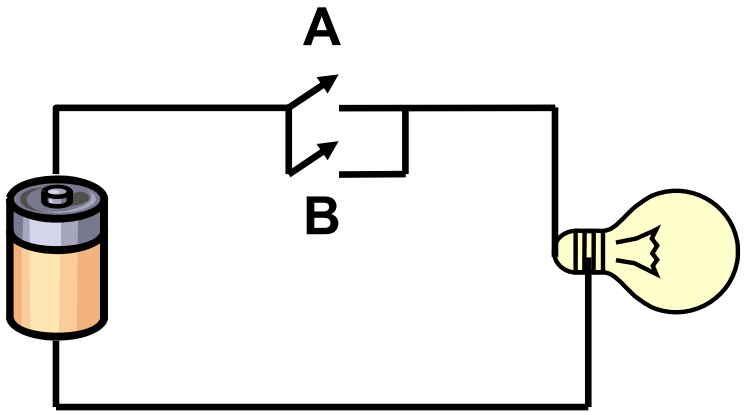


The OR Gate

- This is an **OR** gate
- If either of the two input signals is *asserted*, or both of them are, the output will be *asserted*

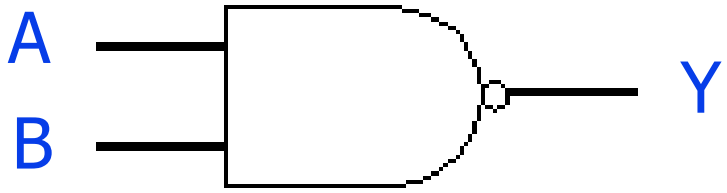
A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1





The NAND Gate

- **The NAND gate is a combination of an AND gate followed by an inverter**
  - **NAND(A, B) → (A AND B)'**



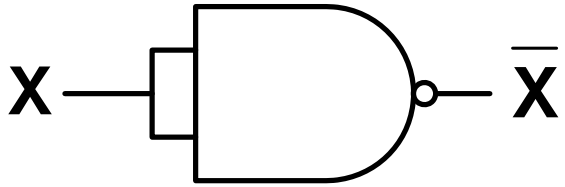
$$Y = \overline{A \cdot B}$$

A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

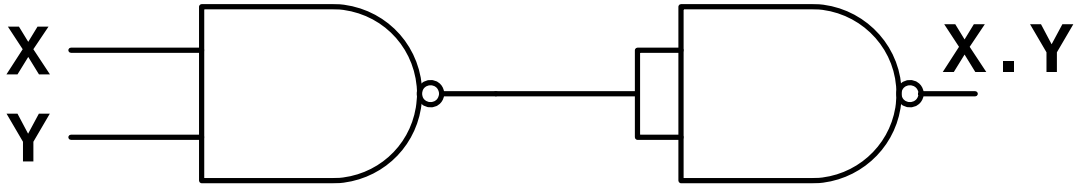
The Universal Property of NAND

- You can implement any function using: NOT, AND, and OR. They represent a **logically complete** set.
- You can use only NAND gates to implement the above three gates. Therefore, NAND alone is a **logically complete** set.

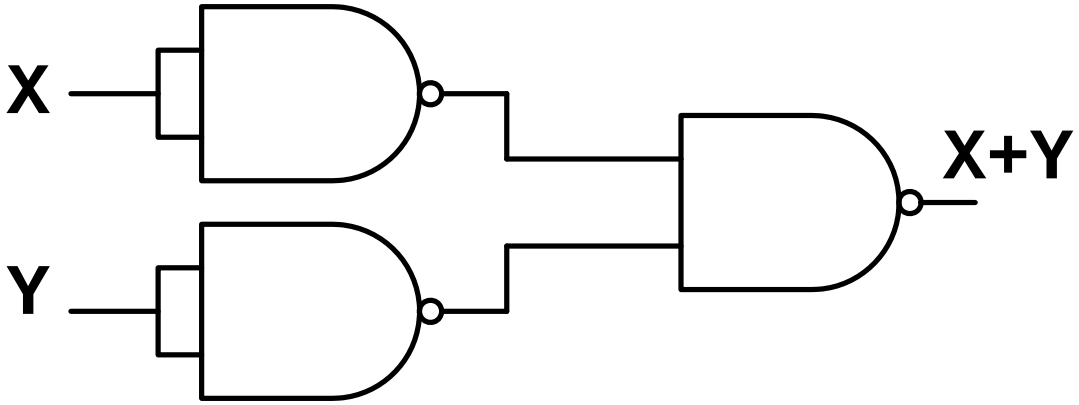
NOT



AND

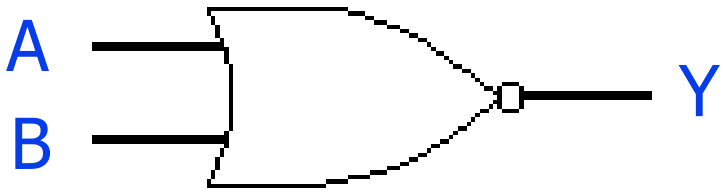


OR



The NOR Gate

- **A NOR gate is a combination of an OR gate followed by an inverter**
  - **NOR(A , B) → (A+B)'**



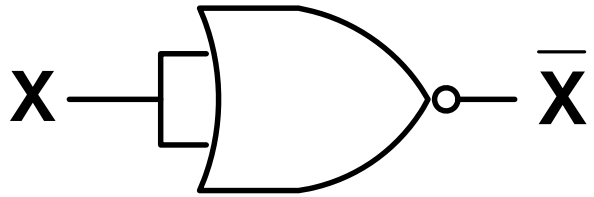
$$Y = \overline{A + B}$$

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

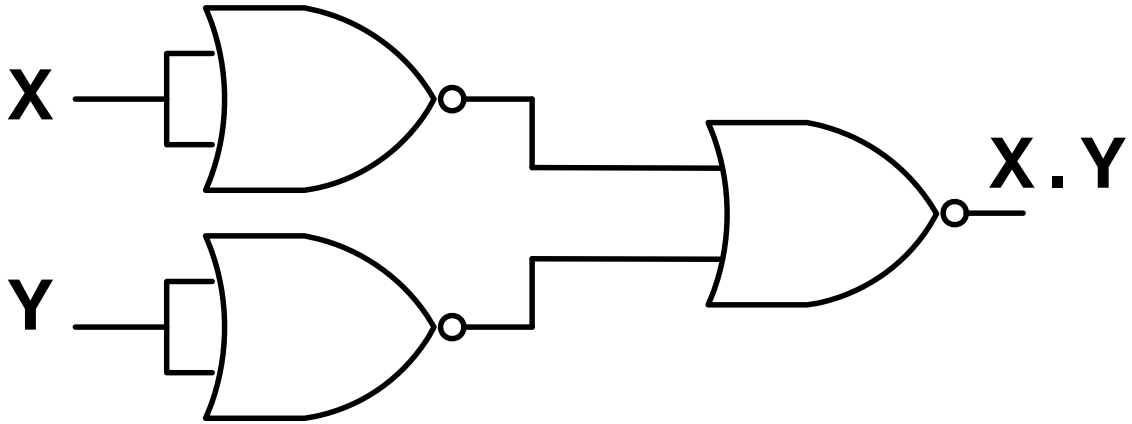
The Universal Property of NOR

- Similarly, you can use only NOR gates to implement NOT, AND, and OR. Therefore, NOR alone is a **logically complete** set.

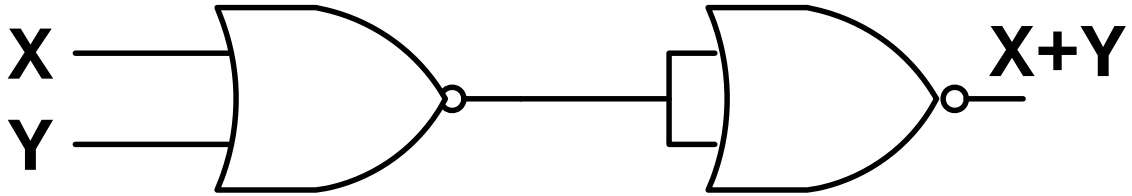
NOT



AND

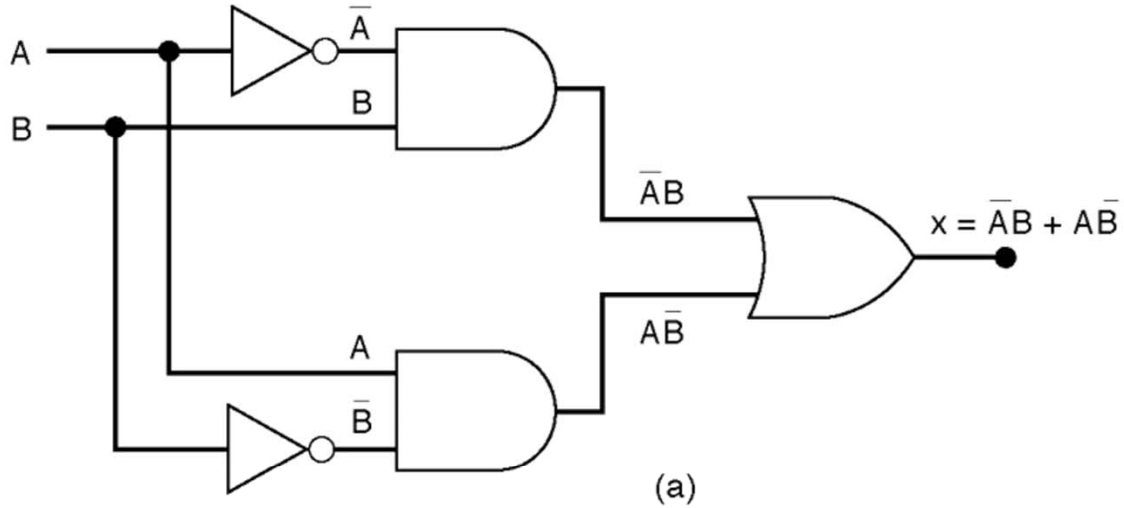


OR



Exclusive-OR and Exclusive-NOR Circuits

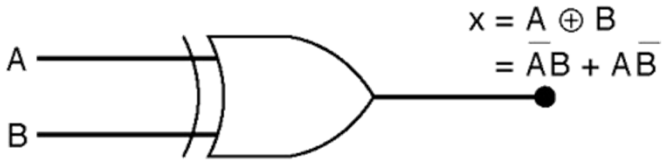
Exclusive-OR (XOR) produces a HIGH output whenever the two inputs are at opposite levels



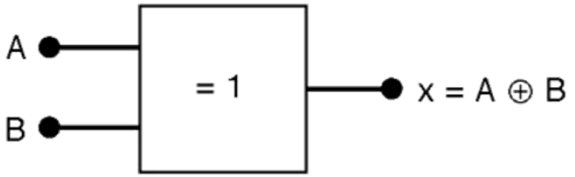
(a)

A	B	x
0	0	0
0	1	1
1	0	1
1	1	0

XOR gate symbols



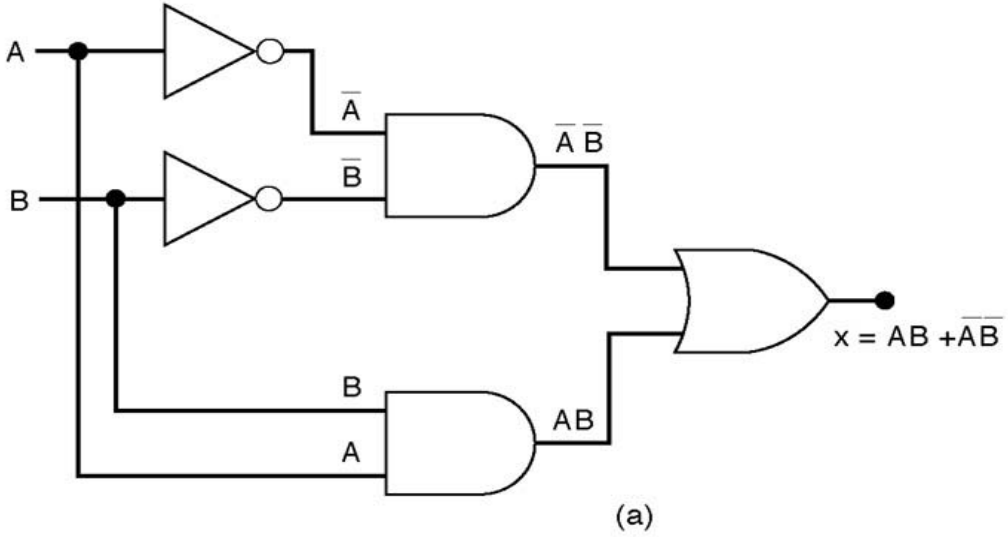
(b)



(c)

Exclusive-NOR Circuits

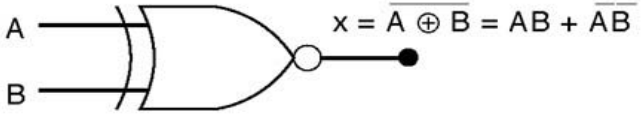
Exclusive-NOR (XNOR) produces a HIGH output whenever the two inputs are at the same level



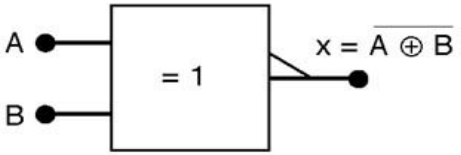
A	B	x
0	0	1
0	1	0
1	0	0
1	1	1

(a)

XNOR gate symbols



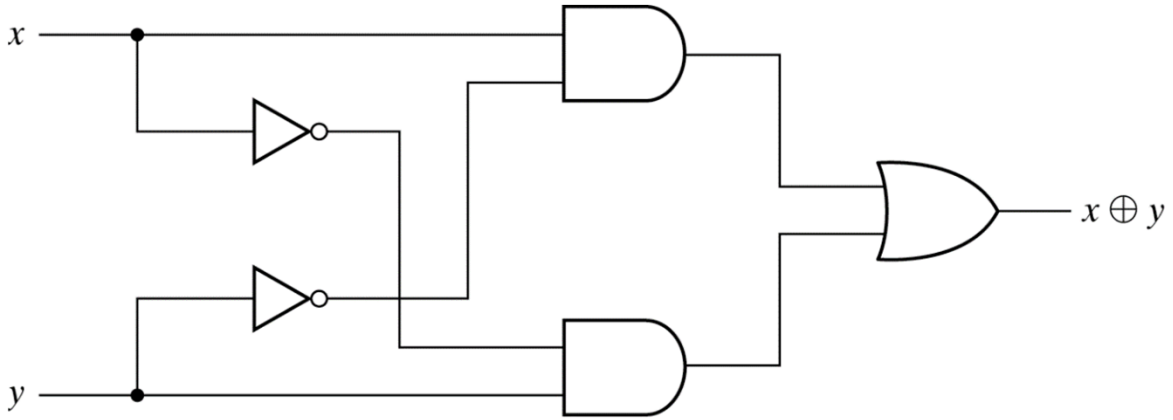
(b)



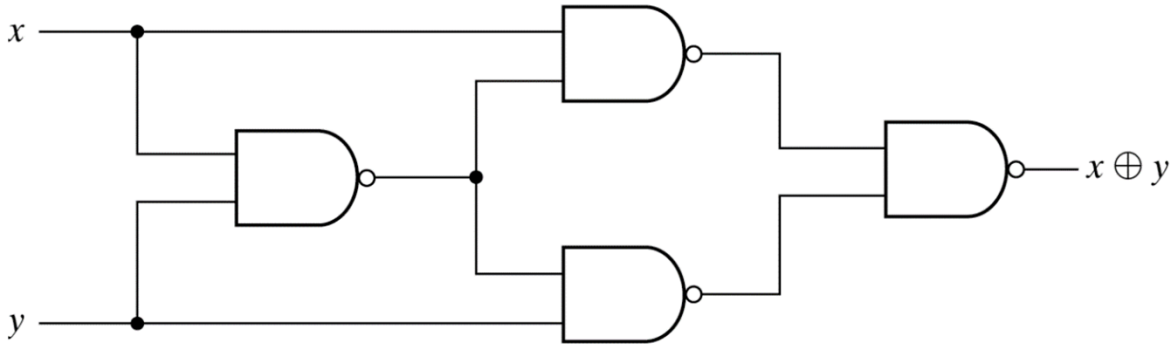
(c)

XOR Function

XOR function can also be implemented with AND/OR gates (also NANDs)



(a) With AND-OR-NOT gates

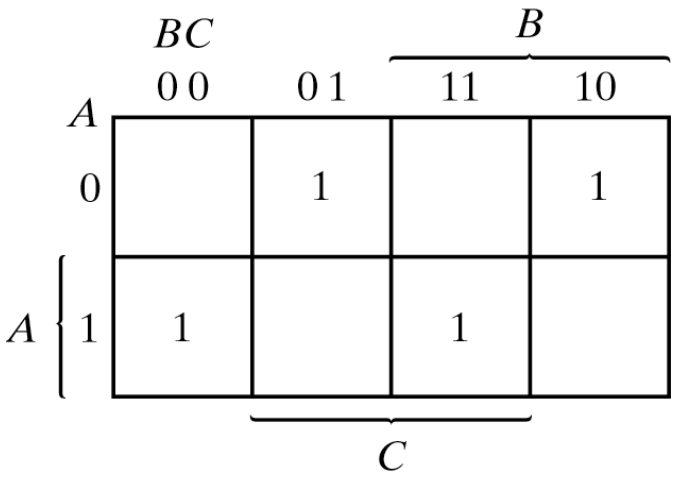


(b) With NAND gates

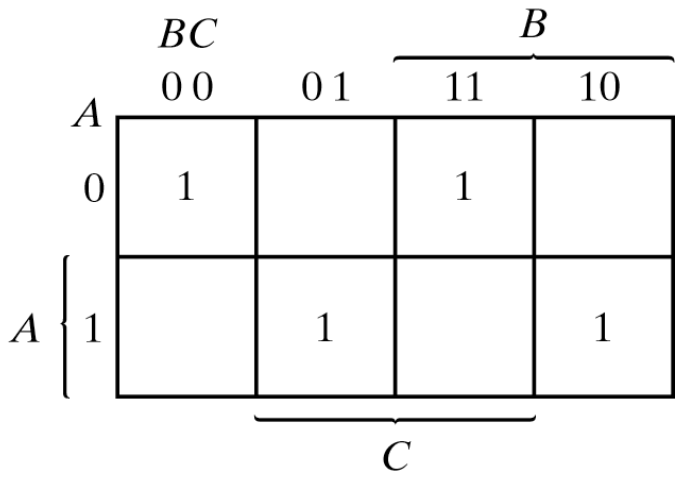
Fig. 3-32 Exclusive-OR Implementations

XOR Function

- Even function – even number of inputs are 1
- Odd function – odd number of inputs are 1



(a) Odd function
$F = A \oplus B \oplus C$

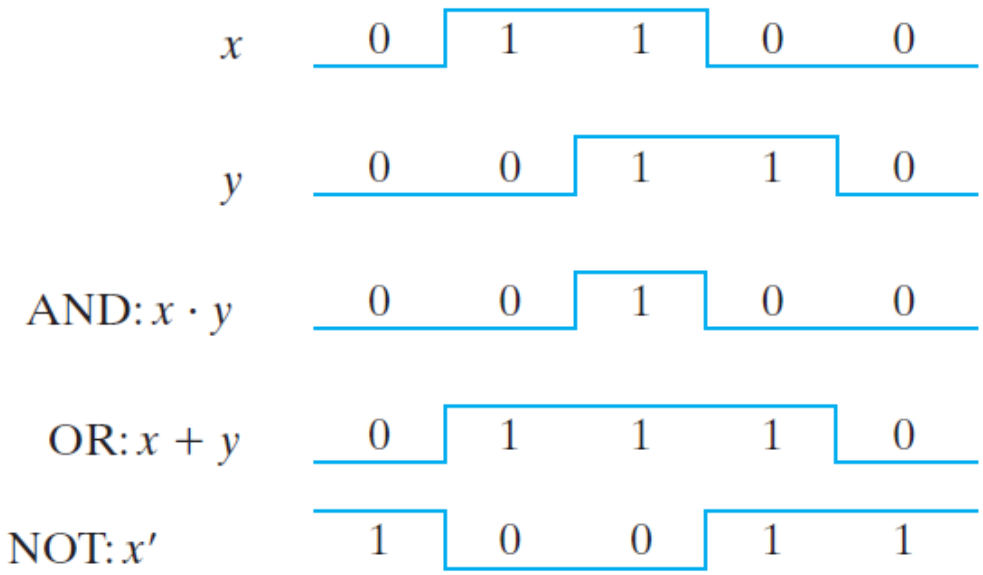


(a) Even function
$F = (A \oplus B \oplus C)'$

Fig. 3-33 Map for a Three-variable Exclusive-OR Function

Describing Circuit Functionality: Waveforms

- **Waveforms provide another approach for representing functionality**
- **Values are either high (logic 1) or low (logic 0)**
- **Can you create a truth table from the waveforms?**

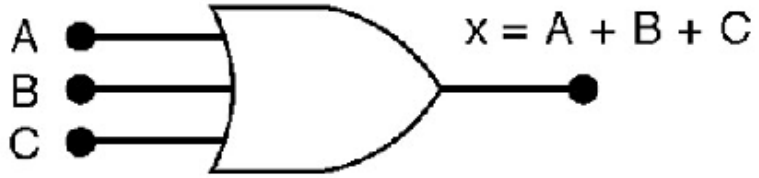


AND Gate

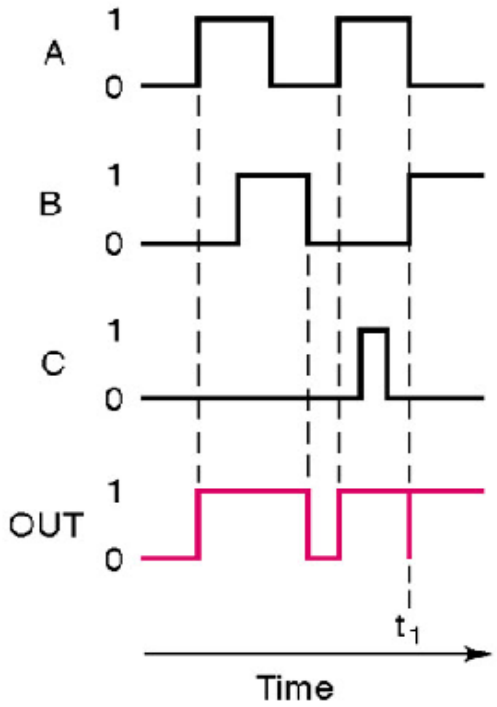
x	y	f
0	0	0
0	1	0
1	0	0
1	1	1

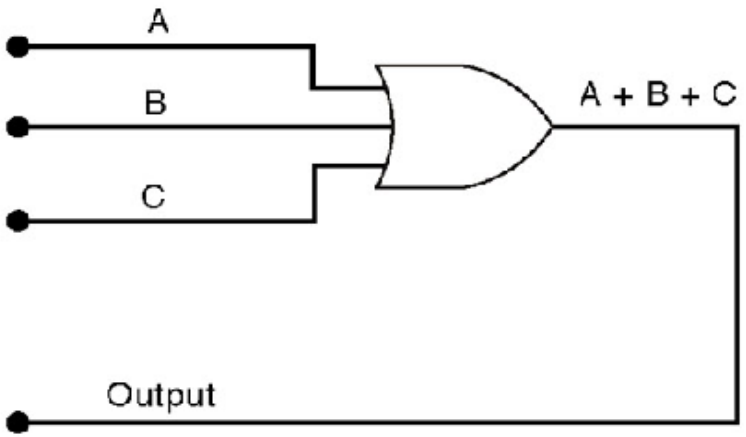
Consider Three-input Gates

3 Input OR Gate



A	B	C	x = A + B + C
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1





Boolean Algebra

- **Useful for identifying and *minimizing* circuit functionality**
- **Identity elements**
  - **a + 0 = a**
  - **a • 1 = a**
  - **0 is the identity element for the + operation**
  - **1 is the identity element for the • operation**
- **The Complement: for every element 'a', there exists a unique element called a' (or ā) (complement of a) such that :**
  - **a + a' = 1**
  - **a • a' = 0**

George Boole (1815 - 1864)

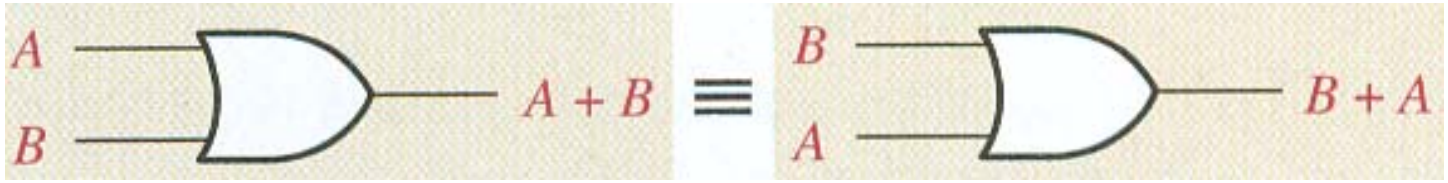
- Father of Boolean algebra
- Boole's system (*detailed in his 'An Investigation of the Laws of Thought, on Which Are Founded the Mathematical Theories of Logic and Probabilities', 1854*) was based on a binary approach, processing only two objects - the yes-no, true-false, on-off, zero-one approach.
- Surprisingly, given his standing in the academic community, Boole's idea was either criticized or completely ignored by the majority of his peers.
- Eventually, one bright student, **Claude Shannon** (1916-2001), picked up the idea and ran with it.

© 2006 Pearson Education, Inc., "Digital Fundamentals", 9/e by Floyd

Laws of Boolean Algebra

- Commutative Law of ORing:

A + B = B + A



- Commutative Law of ANDing:

A . B = B . A

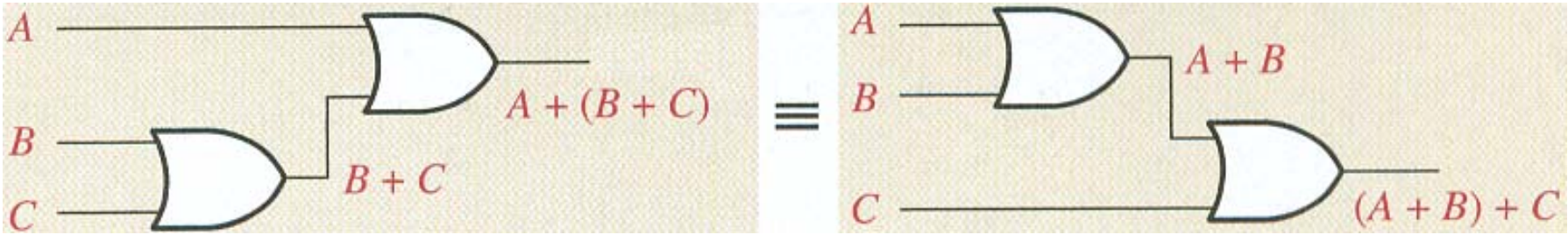


© 2006 Pearson Education, Inc., "Digital Fundamentals", 9/e by Floyd

Laws of Boolean Algebra (Cont'd)

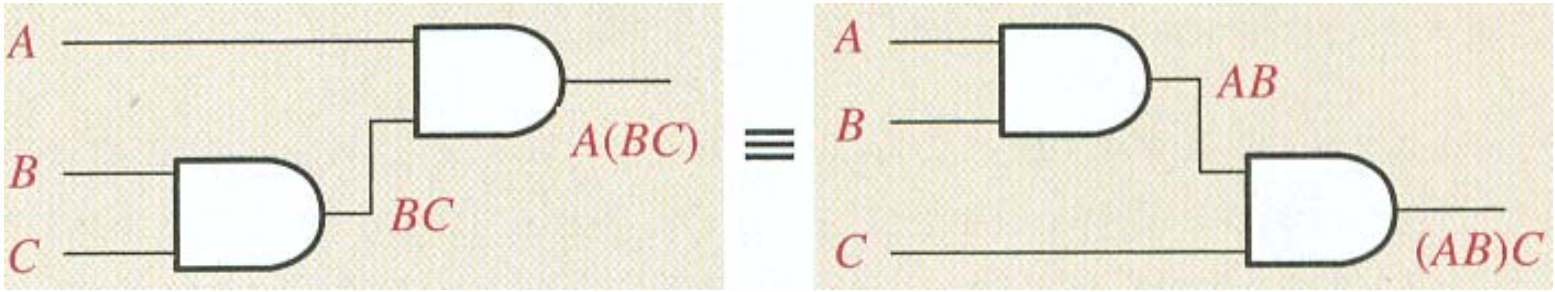
- Associative Law of ORing:

$$A + (B + C) = (A + B) + C$$



- Associative Law of ANDing:

$$A \cdot (B \cdot C) = (A \cdot B) \cdot C$$

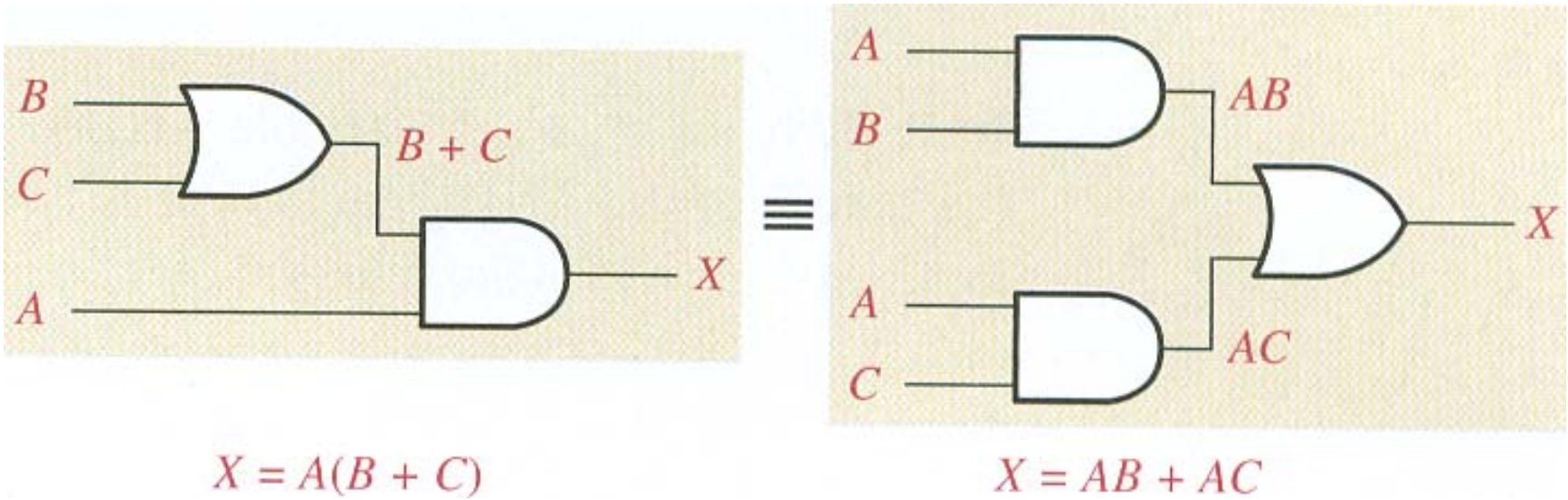


© 2006 Pearson Education, Inc., "Digital Fundamentals", 9/e by Floyd

Laws of Boolean Algebra (Cont'd)

- Distributive Law:
  - *Note:* To simplify notation, the • operator is frequently omitted. When two elements are written next to each other, the AND (•) operator is implied

A(B + C) = AB + AC



© 2006 Pearson Education, Inc., "Digital Fundamentals", 9/e by Floyd



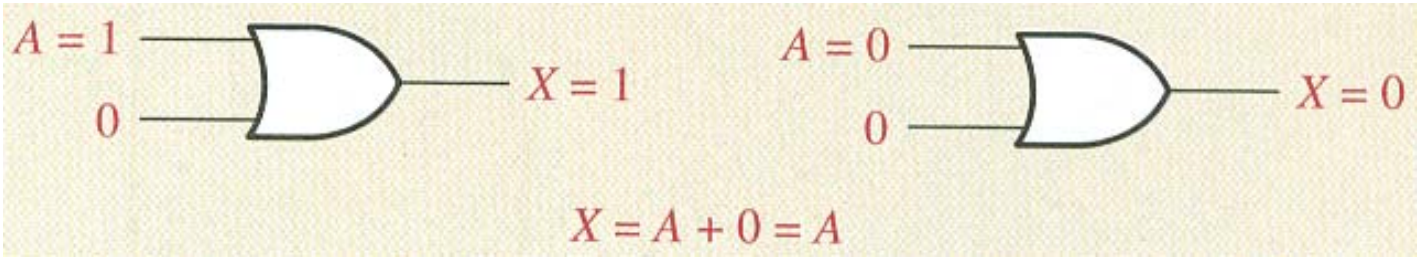
Rules of Boolean Algebra

1. $A + 0 = A$
2. $A + 1 = 1$
3. $A \cdot 0 = 0$
4. $A \cdot 1 = A$
5. $A + A = A$
6. $A + \overline{A} = 1$
7. $A \cdot A = A$
8. $A \cdot \overline{A} = 0$
9. $\overline{\overline{A}} = A$
10. $A + AB = A$
11. $A + \overline{A}B = A + B$
12. $(A + B)(A + C) = A + BC$

© 2006 Pearson Education, Inc., "Digital Fundamentals", 9/e by Floyd

Rules of Boolean Algebra Cont'd

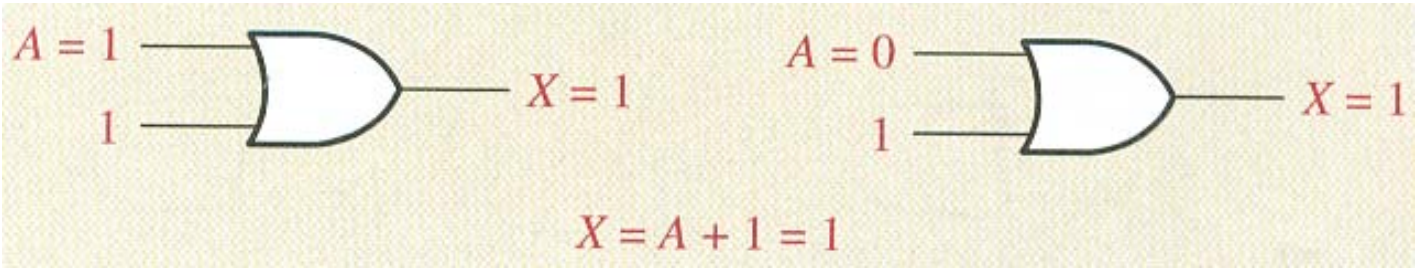
- Rule 1



A	B	X
0	0	0
0	1	1
1	0	1
1	1	1

OR Truth Table

- Rule 2



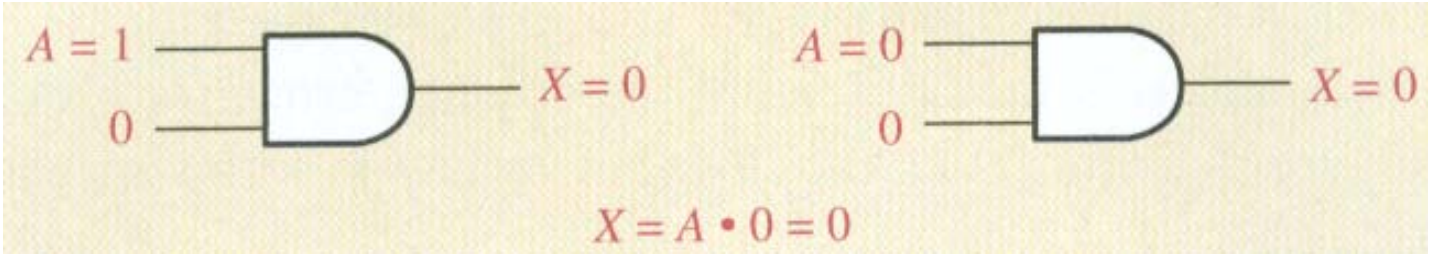
A	B	X
0	0	0
0	1	1
1	0	1
1	1	1

OR Truth Table

© 2006 Pearson Education, Inc., "Digital Fundamentals", 9/e by Floyd

Rules of Boolean Algebra Cont'd

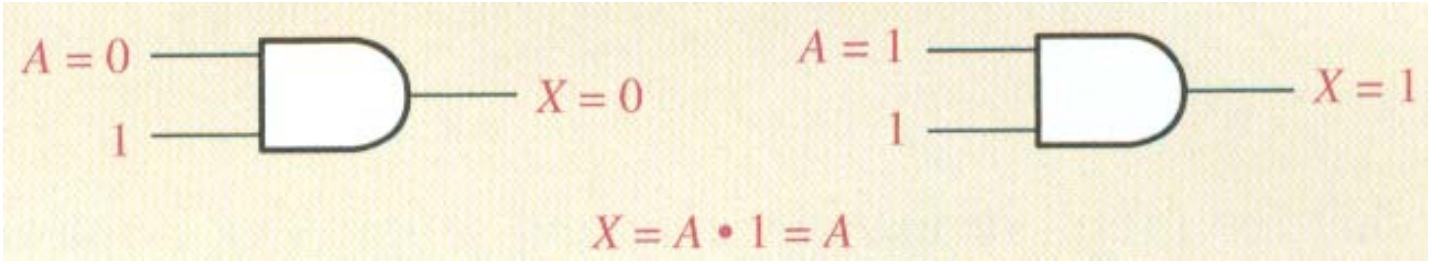
- Rule 3



A	B	X
0	0	0
0	1	0
1	0	0
1	1	1

AND Truth Table

- Rule 4



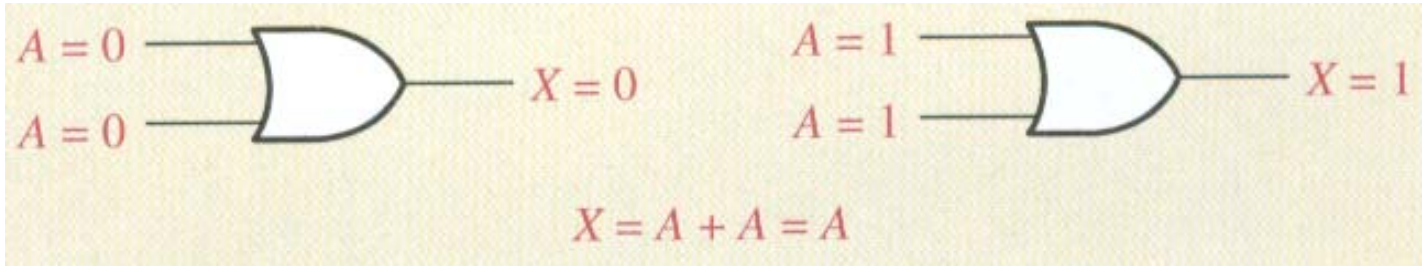
A	B	X
0	0	0
0	1	0
1	0	0
1	1	1

AND Truth Table

© 2006 Pearson Education, Inc., "Digital Fundamentals", 9/e by Floyd

Rules of Boolean Algebra Cont'd

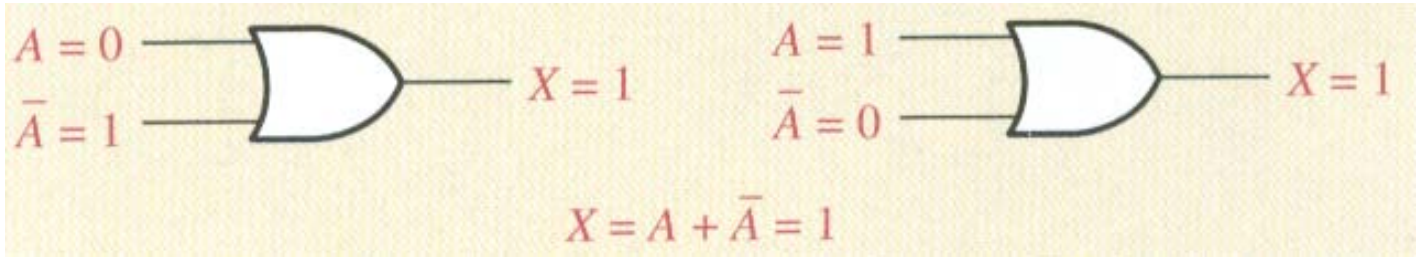
- Rule 5



A	B	X
0	0	0
0	1	1
1	0	1
1	1	1

OR Truth Table

- Rule 6



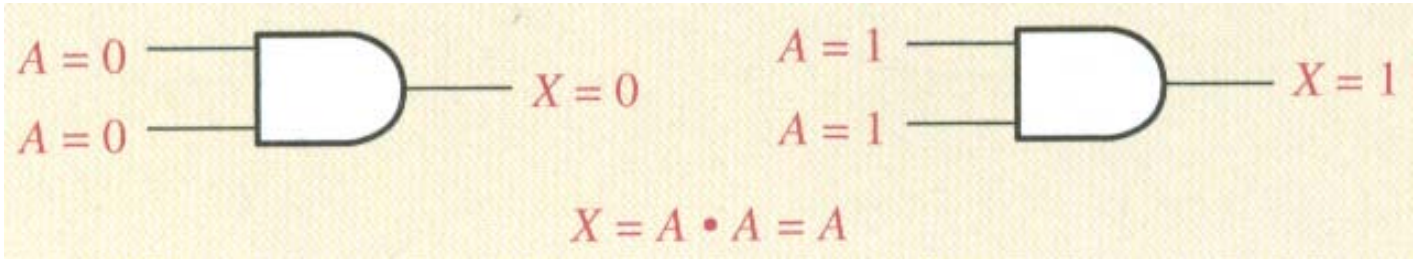
A	B	X
0	0	0
0	1	1
1	0	1
1	1	1

OR Truth Table

© 2006 Pearson Education, Inc., "Digital Fundamentals", 9/e by Floyd

Rules of Boolean Algebra Cont'd

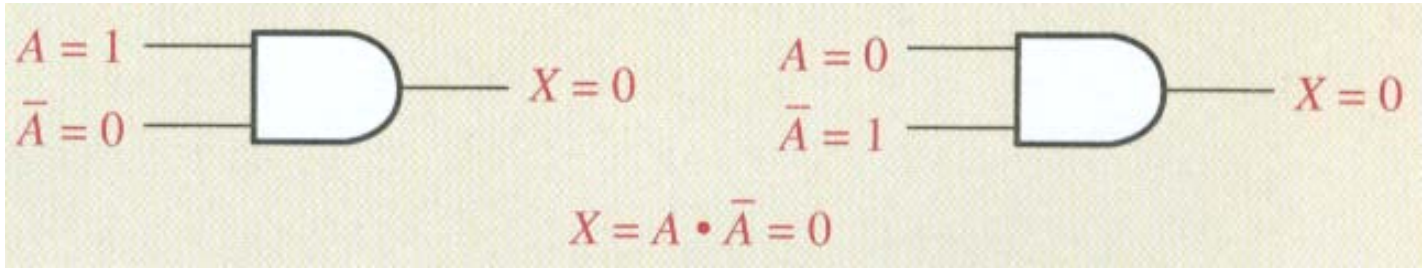
- Rule 7



A	B	X
0	0	0
0	1	0
1	0	0
1	1	1

AND Truth Table

- Rule 8



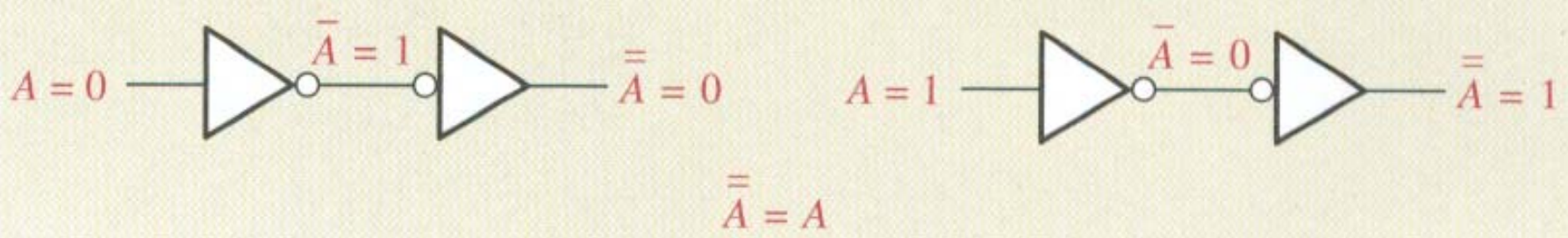
A	B	X
0	0	0
0	1	0
1	0	0
1	1	1

AND Truth Table

© 2006 Pearson Education, Inc., "Digital Fundamentals", 9/e by Floyd

Rules of Boolean Algebra Cont'd

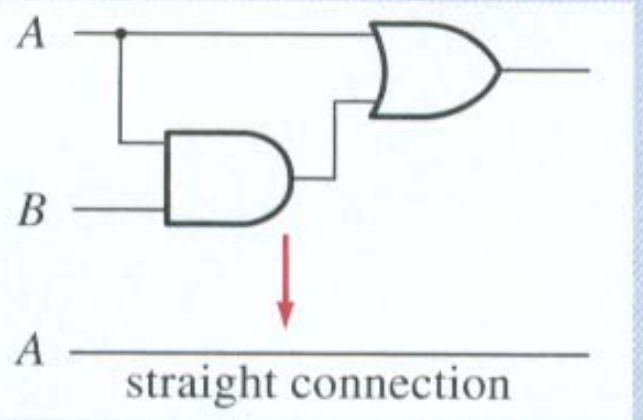
- Rule 9



- Rule 10 (the absorption rule): A + AB = A

A	B	AB	A + AB
0	0	0	0
0	1	0	0
1	0	0	1
1	1	1	1

equal



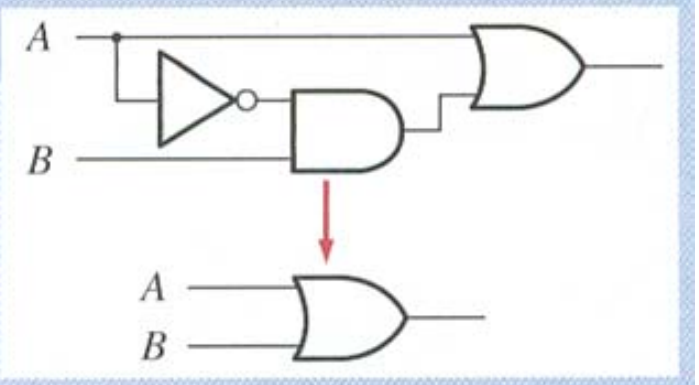
© 2006 Pearson Education, Inc., "Digital Fundamentals", 9/e by Floyd

Rules of Boolean Algebra Cont'd

- Rule 11: $A + \overline{A}B = A + B$

A	B	$\overline{A}B$	$A + \overline{A}B$	A + B
0	0	0	0	0
0	1	1	1	1
1	0	0	1	1
1	1	0	1	1

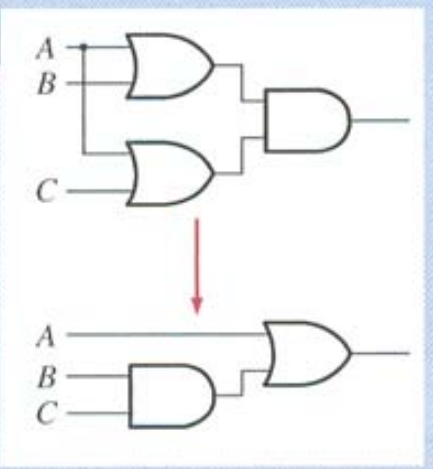
equal



- Rule 12: (A + B)(A + C) = A + BC

A	B	C	A + B	A + C	(A + B)(A + C)	BC	A + BC
0	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0
0	1	0	1	0	0	0	0
0	1	1	1	1	1	1	1
1	0	0	1	1	1	0	1
1	0	1	1	1	1	0	1
1	1	0	1	1	1	0	1
1	1	1	1	1	1	1	1

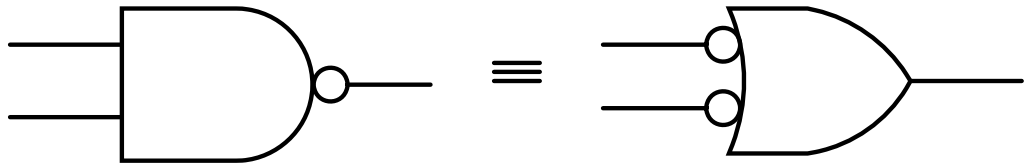
equal



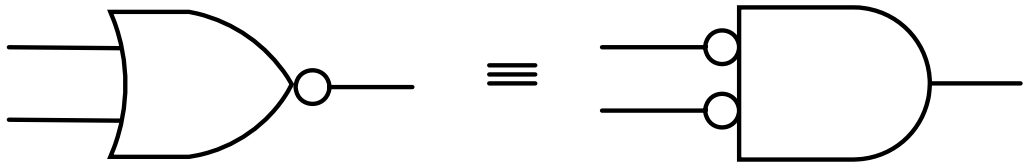
De Morgan's Theorems

(by Augustus De Morgan (1806 - 1871), an English mathematician and logician)

$$\overline{A.B} = \overline{A} + \overline{B}$$



$$\overline{A + B} = \overline{A}.\overline{B}$$



General:

$$\overline{A.B.C.D} = \overline{A} + \overline{B} + \overline{C} + \overline{D}$$

$$\overline{A + B + C + D} = \overline{A}.\overline{B}.\overline{C}.\overline{D}$$

De Morgan's Theorems Example

$$\overline{(A.B+C)\,(A+B.C)}=$$

$$\overline{(A.B+C)}+\overline{(A+B.C)}=$$

$$(\overline{A.B}.\overline{C})+(\overline{A}.\overline{B.C})=$$

$$(\overline{A}+\overline{B}).\overline{C}+\overline{A}.(\overline{B}+\overline{C})=$$

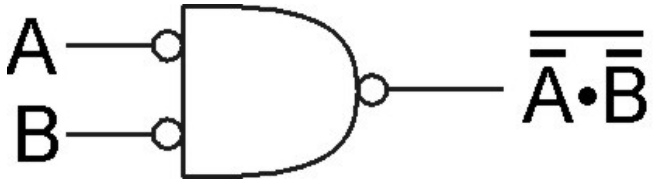
$$\overline{A}.\overline{C}+\overline{B}.\overline{C}+\overline{A}.\overline{B}+\overline{A}.\overline{C}=$$

$$\overline{A}.\overline{C}+\overline{B}.\overline{C}+\overline{A}.\overline{B}$$

Copyright © The McGraw-Hill Companies, Inc.

Converting AND to OR

- Using De Morgan's Theorems, AND can be converted to OR (with some help from NOT)
- Consider the following gate:



A	B	$\overline{A}$	$\overline{B}$	$\overline{A} \cdot \overline{B}$	$\overline{\overline{A} \cdot \overline{B}}$
0	0	1	1	1	0
0	1	1	0	0	1
1	0	0	1	0	1
1	1	0	0	0	1

Same as A+B

To convert AND to OR (or vice versa), invert inputs and output.

© 2006 Pearson Education, Inc., "Digital Fundamentals", 9/e by Floyd



Standard Forms of Boolean Expressions

- The sum-of-product (SOP) form

  Example: X = AB + CD + EF

- The product of sum (POS) form

  Example: X = (A + B)(C + D)(E + F)

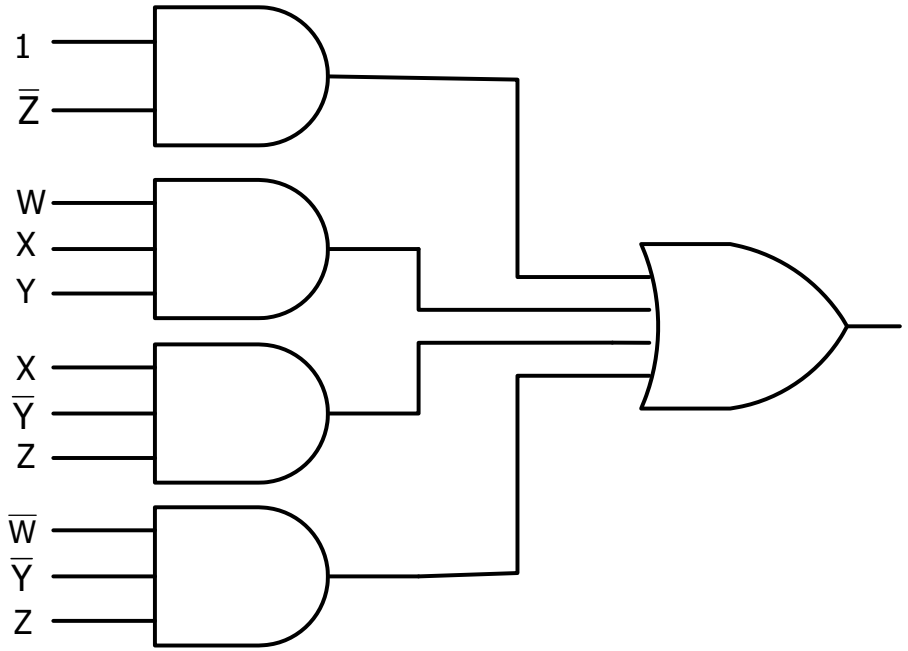
Sum-of-Products Expression

- A *literal* is a variable or the complement of a variable.
  Examples: X, Y, $\overline{X}$, $\overline{Y}$

- A *product term* is a single literal or a logical product of two or more literals. Examples: $\overline{Z}$, $W.X.Y$, $X.\overline{Y}.Z$, $\overline{W}.\overline{Y}.Z$

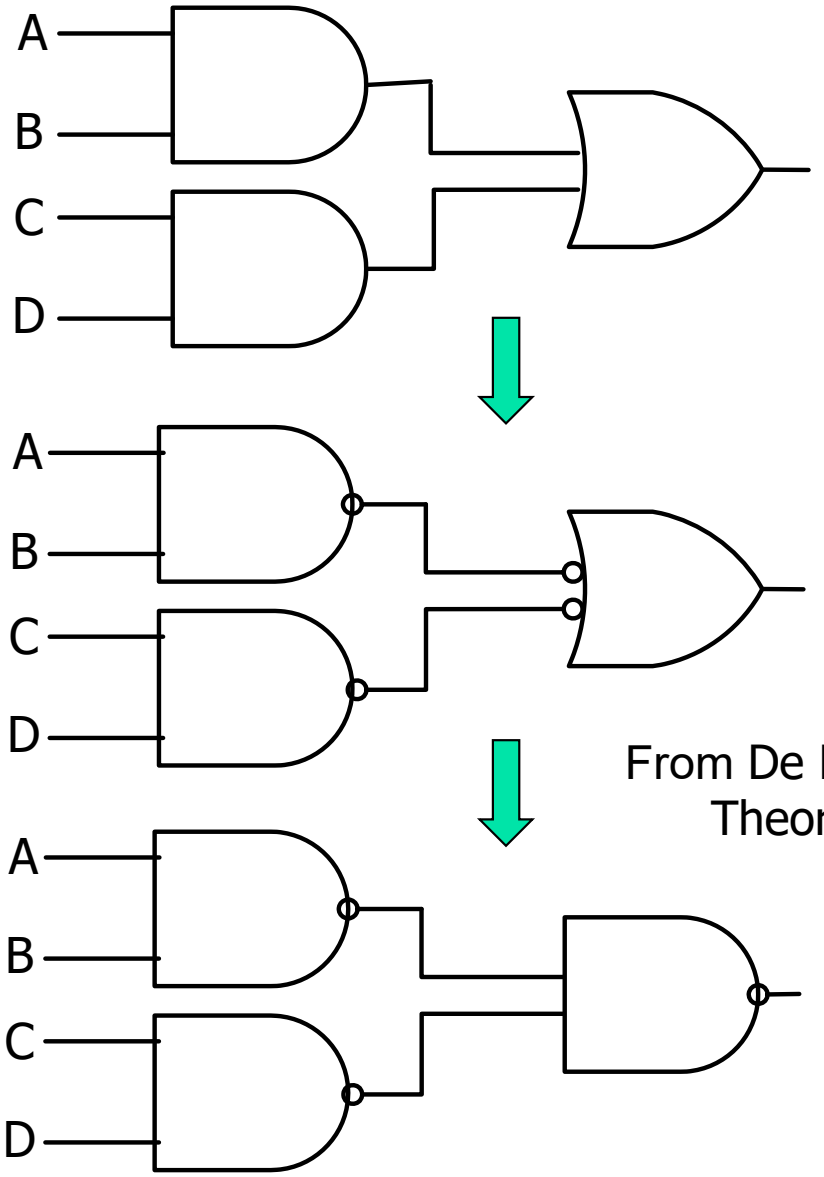
- A *sum-of-products* (SOP) expression is a logical sum of product terms.
  Example: $\overline{Z} + W.X.Y + X.\overline{Y}.Z + \overline{W}.\overline{Y}.Z$

- Every SOP expression can be realized by a two-level circuit containing AND gates followed by an OR gate



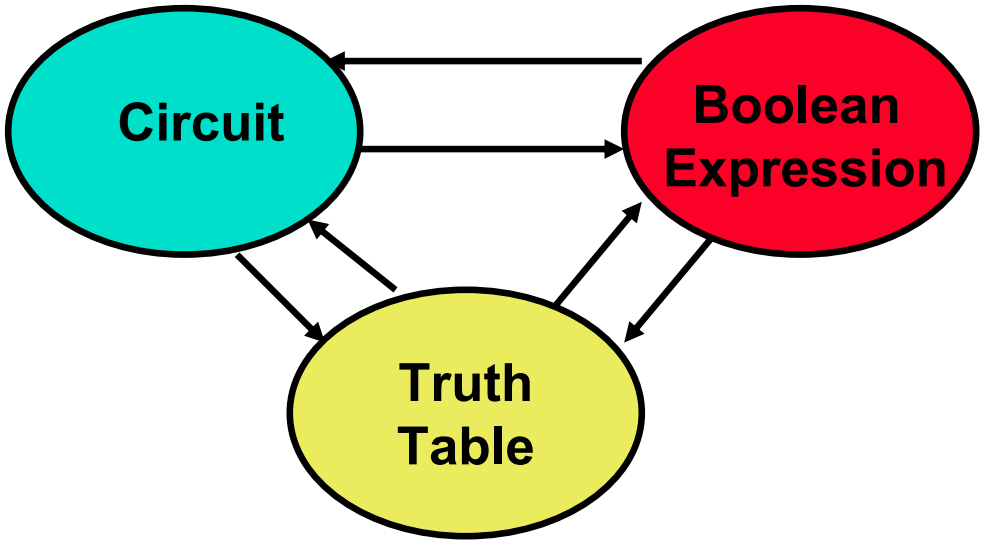
SOP With NANDs

- A SOP expression can be implemented with only NAND gates:



Function Representation Conversion

- **Need to transit between a Boolean expression, a truth table, and a circuit (symbols)**
- **All three formats are equivalent**



Minterm

- For each truth-table row a product term can be defined that evaluates to 1 only when the inputs have the values listed in that row.
- If the product term contains each input variable exactly once it is called a minterm

Row	A	B	C	Minterm
0	0	0	0	$\bar{A}.\bar{B}.\bar{C}$
1	0	0	1	$\bar{A}.\bar{B}.C$
2	0	1	0	$\bar{A}.B.\bar{C}$
3	0	1	1	$\bar{A}.B.C$
4	1	0	0	$A.\bar{B}.\bar{C}$
5	1	0	1	$A.\bar{B}.C$
6	1	1	0	$A.B.\bar{C}$
7	1	1	1	$A.B.C$

From Truth Table to Boolean Expression

- Any logic function can be expressed algebraically by taking the **OR** of all those **minterms** corresponding to the truth-table rows for which the function produces a 1 output.
- *Example:* Majority detector.

Row	A	B	C	Minterm	R
0	0	0	0	$\bar{A}.\bar{B}.\bar{C}$	0
1	0	0	1	$\bar{A}.\bar{B}.C$	0
2	0	1	0	$\bar{A}.B.\bar{C}$	0
3	0	1	1	$\bar{A}.B.C$	1
4	1	0	0	$A.\bar{B}.\bar{C}$	0
5	1	0	1	$A.\bar{B}.C$	1
6	1	1	0	$A.B.\bar{C}$	1
7	1	1	1	$A.B.C$	1

$R=\bar{A}.B.C+ A.\bar{B}.C+ A.B.\bar{C}+ A.B.C$

- Alternate forms:

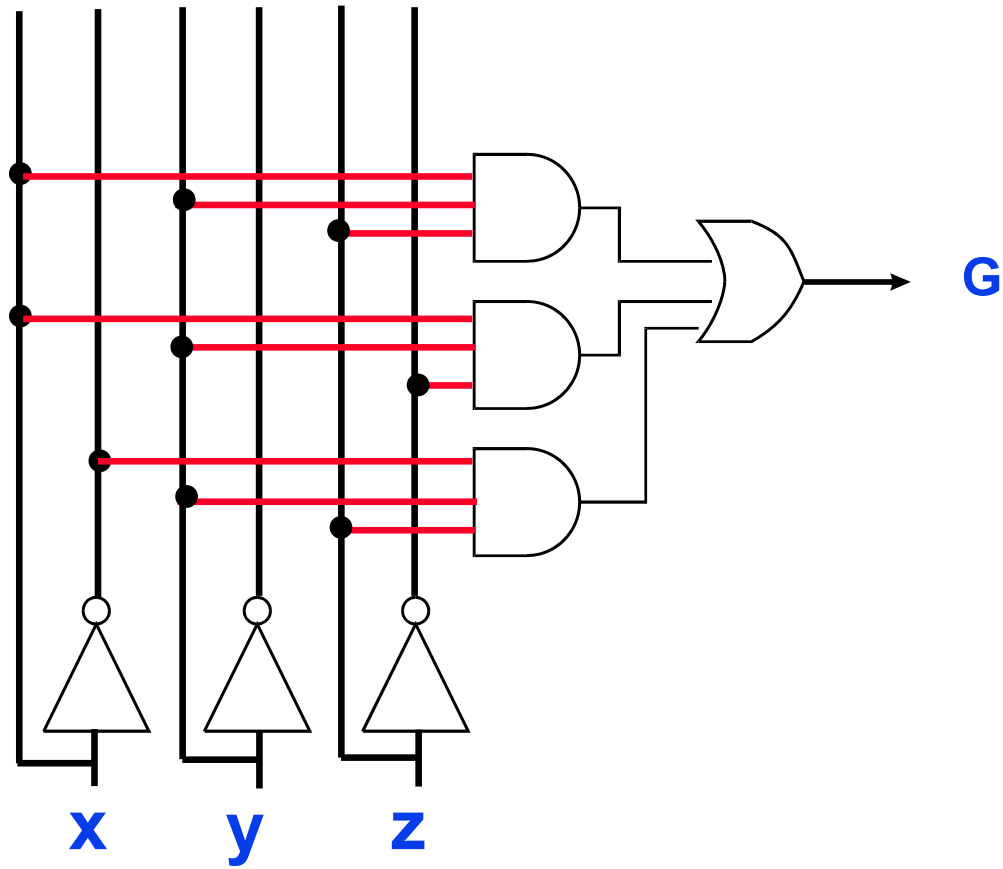
$R = m_3 + m_5 + m_6 + m_7$

$R = \Sigma (3, 5, 6, 7)$

Converting to a Circuit

- **Number of 1's in truth table output column equals AND terms for Sum-of-Products (SOP)**

x	y	z	G
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1



G = xyz + xyz' + x'yz

Canonical Form of a Function

- A form is *canonical* صيغة قياسية if representation of a function in this form is unique.

- Examples:
  - Truth table representation of a function.
  - Sum of minterms representation of a function.

Canonical Form: Simple Example

F(A, B) = A + B — **OR operation**

F = 1 when A = 1 or B = 1 (or both).

We will write the SAME function in two forms:

Ordinary: A + B

Canonical: Sum of Minterms

Step 1 — Truth Table

List all possible values of A and B.

A	B	F = A + B
0	0	**0**
0	1	**1**
1	0	**1**
1	1	**1**

The output is 1 in rows 1, 2 and 3.

Step 2 — Sum of Minterms

Take only the rows where F = 1.

Row 1	Row 2	Row 3
A = 0, B = 1	A = 1, B = 0	A = 1, B = 1
A'B	**AB'**	**AB**

$$F = A'B + AB' + AB$$

$$F(A, B) = \Sigma m(1, 2, 3)$$

Key Takeaway

Ordinary Form: F = A + B

Canonical Form: F = A'B + AB' + AB

Both forms give exactly the same output.

1. الصيغة القياسية= تجمع كل الحالات التي يكون فيها الناتج

Boolean vs. Ordinary Algebra

- Differences
  - Distributivity of + over • holds for Boolean, but not for ordinary algebra

$$x+(y \cdot z) = (x+y) \cdot (x+z)$$

  - Boolean algebra does not have inverse elements for + or •

    Thus, no subtraction or division operators

  - Complement is not defined in ordinary algebra